02465 Project: Part 1

Tue Herlau
tuhe@dtu.dk

March 2, 2026

Formalities

- The deadline for this report is **Thursday 5th March, 2026** before 23:59.
- Submission of reports happen on DTU learn
- You can work in groups of 1, 2 or 3 students (but not 4)
- Collaboration policy: It is not allowed to collaborate with other groups on this project, except for discussing the text of the project with teachers and students enrolled on the course this semester. It is not allowed to communicate (or make available) solutions or parts of solutions to the project to other people. It is not allowed to use solutions from previous years, or solutions found on the internet or elsewhere.
- You can freely use code from the *exercises* when you solve the project, for instance the dynamical programming algorithm.
- Your overall evaluation will be based on your written answers and your UNITGRADE score. They will be weighted based on an assessment of the required work.

Preparing the hand-in:

Hand in these three files (please do not hand in a `.zip` file as this confuses DTU learn):

A `.tex` file with your written answers: Prepared this by modifying the template in `irlc/project1/Latex/02465project1_handin.tex` . Simply write your answers where it says **YOUR SOLUTION HERE**. I recommend keeping the layout as it is.

A `.pdf` file corresponding to this `.tex` file

A `.token` file containing your python-solutions: Generate this file by running the script `irlc/project1/project1_grade.py` . It is very important you do not modify this file.

In order to hand in the assignment on DTU Learn you must be part of a group. You can join a group on DTU learn from *My Course* → *Groups*.

Contribution table

DTUs exam rules require that each students contribution to the report is clearly specified. Therefore, for each element in the report, specify which student was responsible for it in the table in the template. **A report must contain this documentation to be accepted**. The responsibility assignment must be individualized. This means:

- For reports made by 3 students: Each section must have a student who is 40% or more responsible.
- For reports made by 2 students: Each section must have a student who is 60% or more responsible.

This is an external requirement. Ask me if you have any questions.

Code hand-in:

- Please keep the structure of the `irlc`-folder. All of your code which is specific to this report should be in the `irlc/project1/` directory. Solutions which use code outside the `irlc` folder cannot be verified and therefore cannot be evaluated.
- If you wish to use additional third-party libraries please discuss them with me first to ensure you are on the right track.
- Breaking or tampering with the UNITGRADE framework, for instance by reporting a false number of points or making your solution unverifiable, is potentially cheating. Code which is obfuscated to the point of being unreadable cannot be evaluated.
- This is not a programming course: Strange, long, undocumented, or downright disturbing solutions will be evaluated simply based on whether they work or not.

1 The kiosk (`kiosk.py`)

In this problem, you take the role of a blaster salesman on the desert planet of Tatoine. You sell blasters to Jawas and Tusken Raiders and, as we will see, this is a surprisingly well-regulated line of work with some unique challenges.

Your job in this problem is to determine how many blasters to buy each day in order to maximize your expected profit. Let's first establish the basic rules:

- You can have $0, 1, 2, \ldots, n_s$ blasters in the kiosk.
- You can order $0, 1, 2, \ldots, n_o$ blasters to restock your inventory
- The cost of ordering a single blaster is $c_o = 1.5$ credits
- The sale price of a blaster is $s_p = 2.1$ credits
- You will plan on a horizon of $N = 14$ days

Running a kiosk resembles the inventory-control problem which we saw in [Her25, section 5.1.2]. Each day, starting in day $k = 0$, it goes as follows:

- Very early in the morning you have an initial inventory stock x_0
- Based on this, you submit an order of a given number of blasters u_0 to the orbital merchant ship
- The number of blasters you ordered are delivered before your shop opens
- Customers will attempt to buy a number w_0 of blasters during the day
- Based on the initial inventory, the number of ordered blasters, and the number of purchases you obtain a daily reward[1],

$$g_0(x_0, u_0, w_0) = c_o u - s_p \min(x_0 + u_0, w_0)$$

  , and a final inventory[2] $x_1 = f_0(x_0, u_0, w_0)$
- The process is repeated with $k = 1, 2, ...$

As an example, suppose over the full $N = 14$-day period you sell a total of 6 blasters and buy 5. The total (accumulated) profit will then be

$$6 \times \{\text{sale price}\} - 5 \times \{\text{ordering price}\} = 6 \times 2.1 - 5 \times 1.5.$$

How you should plan will depend on your assumptions. These are going to change from problem to problem as we take more effects of local life into account: We start from a simple model, and then make it more realistic. Therefore, although the problems can be solved independently, I recommend solving the simple problems first and modifying the solutions.

Problem 1 *A basic blaster-business*

To get started, we make the following assumptions:

- The empire does not allow you to store more than $n_s = 20$ blasters overnight for safety reasons. In other words, suppose in the morning you have $x = 15$ blasters, you order an additional 10, and you sell a total of 3. Then in the evening you will have 15+10-3 = 22 blasters, and you have to discard two blasters so the inventory the following morning will be $x' = 20$.
- Discarding blasters does not cost anything
- Your cost-function is solely determined by your profit; the more profit, the better
- you can order up to $n_o = 15$ blasters at a time

In addition to this, suppose that Tusken raiders always buy blasters 3 at a time. Therefore, the chance there is a demand of 0 blasters in a day is 30%, 3 blasters in a day is 60% and 6 blasters in a single day is 10%; all other demands can be ruled out.

Given this information, formulate the blaster-business as standard decision problem below by completely defining it mathematically. Your formulation must be self-contained; i.e., sufficient to implement the environment.

[1] The reward takes into account you cannot sell more blasters than you got!

[2] Subject to the same restrictions as the inventory control problem, i.e., it cannot be negative etc.!

Answer:
YOUR SOLUTION HERE Include the following elements:

$$N = 14 \tag{1}$$
$$\textbf{for } k = 0, \dots, N\textbf{:} \quad \mathcal{S}_k = \dots \tag{2}$$
$$\textbf{for } k = 0, \dots, N-1\textbf{:} \quad \mathcal{A}_k(x_k) = \dots \tag{3}$$
$$\textbf{for } k = 0, \dots, N-1\textbf{:} \quad f_k(x_k, u_k, w_k) = \dots \tag{4}$$
$$\textbf{for } k = 0, \dots, N-1\textbf{:} \quad g_k(x_k, u_k, w_k) = \dots \tag{5}$$
$$g_N(x_N) = \dots \tag{6}$$
$$\textbf{for } k = 0, \dots, N-1\textbf{:} \quad P_W(w_k | x_k, u_k) = \dots \tag{7}$$

Problem 2 *Warmup*

Now that we have gotten this far, complete the functions `warmup_states()` and `warmup_actions()` which should return $\mathcal{S}_0$ and $\mathcal{A}(x_0)$ respectively. Since $\mathcal{A}(x_0)$ is independent of x_0, you can ignore the x_0-value.

Info:

- These should be around one line each – just return the state and action spaces as sets
- Read the description of the problem above (or look at the previous exercise)
- Look at the inventory-control environment for inspiration

Problem 3 *Manually computing* J_{N-1}

Consider the expected optimal cost-to-go just before the last action is taken, $J_{N-1}(x_{N-1})$. Suppose x_{N-1} corresponds to a completely full inventory. Calculate the value of $J_{N-1}(20)$ below and explain your calculation.

Answer:
YOUR SOLUTION HERE The last step of the DP algorithm gives:

$$J_{N-1}(20) = \min_{u_{N-1}} \mathbb{E}_{w_{N-1}} \left[g_{N-1}(20, u_{N-1}, w_{N-1}) + g_N(x_N) \right]$$

Therefore ...

Info:

- Beware of the signs $\pm$!
- The problem is about inserting and reducing, so start by inserting what you know.

Problem 4 *Compute optimal policy and value function*

Implement the function `solve_kiosk_1()` which returns the optimal value function and policy in the usual format. That is:

- Value functions are a list-of-dictionaries such that `J[k][10]` is $J_k(x = 10)$
- Policies are a list-of-dictionaries such that `pi[k][10]` is $\mu_k(x = 10)$ (i.e. the number of blasters to buy if the inventory is $x = 10$ blasters in planning round k)

Info:

- The recommended way to solve this problem is by using the DP algorithm, `DP_stochastic`, just like the inventory-control environment. You can call the code you used in the exercises.
- If you take this approach, you should implement a DP-model corresponding to the problem
- The format of `J` and `pi` discussed above is compatible with the output of the DP algorithm
- Check your results using UNITGRADE

Problem 5 *Kiosk2*

As it turns out, the previous plan was way to to naive and failed to take two important factors into account

- The demand for blasters actually resemble a binomial distribution. The chance that $w = 0, \ldots, n_s$ blasters are bought in a given day is

$$p(w) = \binom{n_s}{w} p^w (1-p)^{n_s - w}.$$

  Where n_s is the storage space. Using historical data you determine that $p = \frac{1}{5}$.

- When you dispose of excess blasters (i.e., blasters that you are not allowed to store over night) you have to obey the pesky imperial environmental protection act regarding safe handling of dedlanite and bla bla bla. Anyway, it costs 3 credits to dispose of a *single* excess blaster. This means that:

$$g_k(x, u, w) = c_o u - s_p \min(x + u, w) + 3\{\text{Excess blasters}\}$$

  In the example from problem 1, we had $x = 15$ blasters, ordered $u = 10$, sold $w = 3$, and that means the *excess blasters* were $x + u - w - 20 = 2$.

Implement these rules in `solve_kiosk_2()` and check how it affects your profits.

Info:

- You probably need to change two functions
- `scipy.stats` has a build-in binomial distribution function. Alternatively, just implement the function yourself using the above equation.
- Manually check if you compute the binomial probabilities correctly; this is an easy way to avoid one potential source of problems
- Check your results using UNITGRADE

1.1 Explaining policies

Problem 6 *Policy explanation continued*

The two policies are plotted in fig. 1. They differ as to how many blasters should be bought on day $N - 1$ (i.e., the last day where a decision is made) assuming the inventory is empty. Provide a mathematical argument to show how many blasters the first policy prefers to buy on the last day given the inventory is empty, i.e. compute $\mu_{N-1}(0)$ manually.

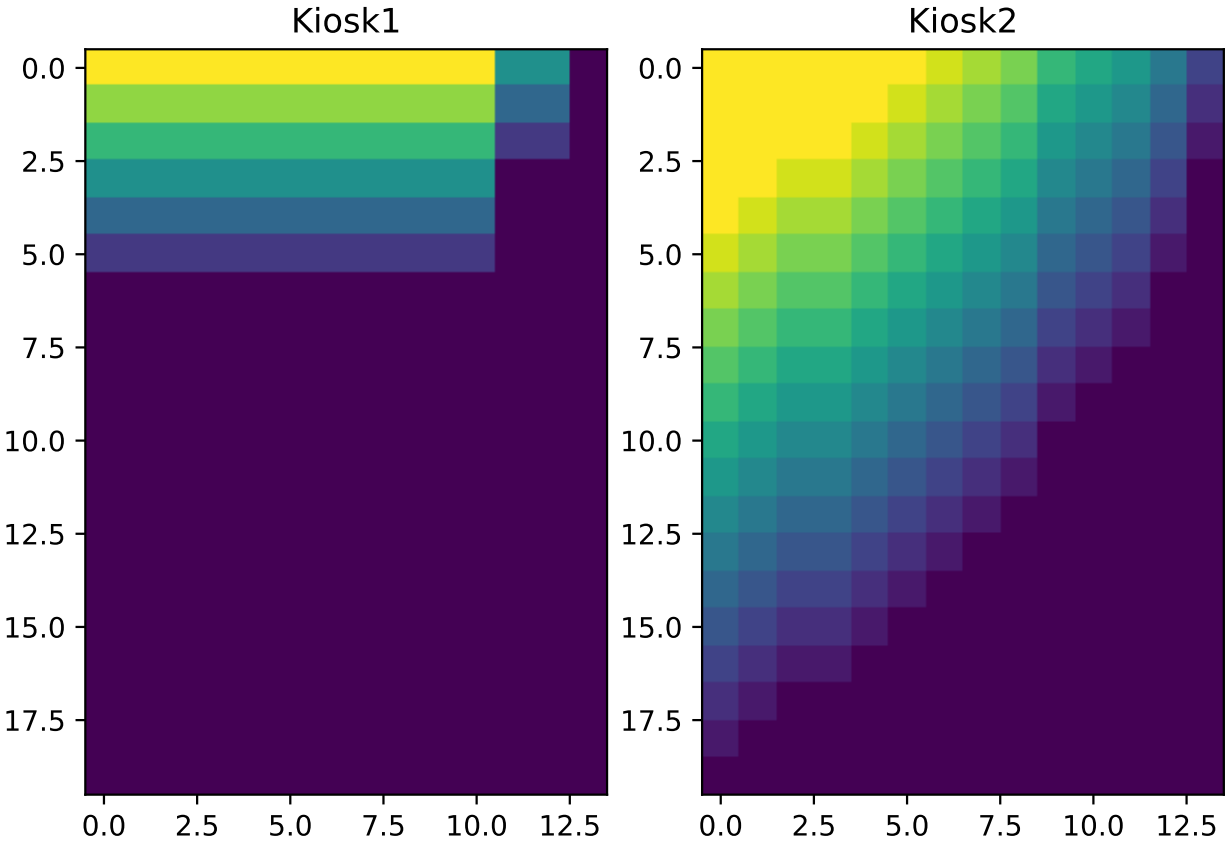

Figure 1: Plot of policies for the two first questions (kiosk1 and kiosk2)

Answer:

$$\mu_{N-1}(0) = \arg\min_u \mathbb{E}_w \left[g_k(0, u, w) + g_N(f(0, u, w)) \right] = \cdots$$

YOUR SOLUTION HERE

Info:

- Remember, this is what you code does right now; it just does not tell you the intermediate calculations.

2 Avoid the droid (pacman.py)

R2D2, who in this problem bears a remarkable resemblance to pacman, must first find the data-discs with the plans for the death-star (these are illustrated as small dots) while avoiding the blue patrol ghost-droids. Since this is rather important, R2D2 must determine the absolutely optimal plan which we will do using dynamical programming. To solve the problem, I recommend looking at the online documentation.

Your job is to help R2D2 by carrying out a sequence of increasingly difficult tasks culminating in the full solution. It is recommended that you make sure each task is completed correctly before you progress to the next one (use the UNITGRADE test-scripts). The first tasks are easier, since R2D2 does not have to account for the patrol-droids and just has to find the optimal way to pick up data-discs.

2.1 Getting set up

Maps are specified as strings (see fig. 2) such as

```python
# pacman.py

east = """
%%%%%%%%
% P    .%
%%%%%%%% """
```

The maps are loaded using an openai gym environment. The environments `reset()` method can then be used to get a state corresponding to this map configuration:

```python
    # pacman_demo1.py
    # Instantiate the map 'east' and get a GameState instance:
    env = PacmanEnvironment(layout_str=east, render_mode='human')
    x, info = env.reset() # x is a irlc.pacman.gamestate.GameState object. See the online documentation for more examples.
    print("Start configuration of board:")
    print(x)
    env.close() # If you use render_mode = 'human', I recommend you use env.close() at the end of the code to free up graphics resources.
    # The GameState object `x` has a handful of useful functions. The important ones are:
    # x.A()        # Action space
    # x.f(action) # State resulting in taking action 'action' in state 'x'
    # x.players() # Number of agents on board (at least 1)
    # x.player()  # Whose turn it is (player = 0 is us)
    # x.is_won()   # True if we have won
    # x.is_lost()  # True if we have lost
    # You can check if two GameState objects x1 and x2 are the same by simply doing x1 == x2.
```

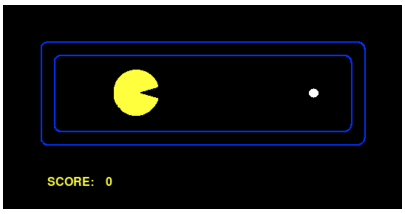

Figure 2: The corridor-map

The state `x` is a bit special as it is actually an instance of a `GameState` object, which means that it contains a number of useful functions to get the available actions and so on. These functions, which are shown in the comment above, are *all* you need to use to complete this project. The above output also shows one of the things we can do with the `GameState`, namely print the game state corresponding to the `GameState`:

```
Start configuration of board:
%%%%%%%%
% <    .%
%%%%%%%%
Score: 0

```

The rest of the code also shows how you can render the `GameState` in a nicer way as shown in fig. 2, and in `pacman_demo2.py` you can play the game yourself.

The first question will help familiarize ourselves with the `Gamestate` by simply walking east to get to the datadisc.

Problem 7 *Go east*

Help R2D2 get to the datadisc by completing the function `go_east`. The function should take a map (a string) as an input, and return a list of `GameState` objects corresponding to the path to the datadisc, starting in the initial configuration of the map. The last `GameState` in the list should correspond to the winning configuration.

You should assume the map has the form of an east-bound corridor (as in fig. 2), but of arbitrary length – R2D2 can therefore solve the problem by just going east until victory.

Info:

- This is supposed to be an easy question. Don't get too creative, just go east!
- Check out the code in `pacman_demo1.py` to see how to instantiate the gamestate `x`. After you have instantiated `x`, experiment with it in the console interpreter
- Use the action-space function `x.A()` to figure out what the east-action is
- Use the next-state function `x.f(action)` to move
- Use `print(x)` whenever you are confused about what `x` is
- Put all the `x`'s in a list and return it
- Make sure the function works for arbitrary-length corridors (but still, you only have to consider the east-bound direction)
- Use the UNITGRADE test script to debug your code. Remember you can re-run individual tests to limit the junk-output.

2.2 No droid planning

To apply dynamical programming, we must first determine the $N+1$ state-spaces $\mathcal{S}_0, \ldots, \mathcal{S}_N$. This question will break this problem into smaller tasks which, once complete, will make applying DP trivial. This also means the task formulation may seem a little counter-intuitive at first.

Problem 8 *Predict consequence of actions*

First, R2D2 must know the consequences of a single action. Do this by completing the `p_next(x,u)` function, which takes a `GameState` and an action as input, and return a dictionary which has the next possible states you can transition into when you take action `u`) as keys, and the probability of this transition as the value. We want this function to compute:

$$p(x_{k+1}|x_k, u_k)$$

For all possible next states x_{k+1} when the game is in state x_k and pacman (the player) selects action u_k, however, since we don't consider any ghosts yet the problem is deterministic.

Info:

- The problem is deterministic so $P(x_{k+1}|x_k, u_k) = 1$ and the next state x_{k+1} is `x.f(u)`.
- You should therefore just return `{ x.f(u) : 1 }`
- Your solution will likely just be a single, short line

We can now construct the state-spaces $\mathcal{S}_0, \ldots, \mathcal{S}_N$. This will be done using a single function, which should therefore return a *list of lists of states*[3], such that the k'th element is $\mathcal{S}_k$, which corresponds to the states R2D2 can reach in exactly k moves starting from the initial state

Problem 9 *Possible future states*

Complete the function `get_future_states(x, N)`, which return a list of length $N + 1$ such that the k'th element correspond to $\mathcal{S}_k$ (the states R2D2 can reach in k steps starting from `x`.

Info:

- I recommend using `p_next` when solving this problem
- The first set $\mathcal{S}_0$ is just the singleton list `[x0]`
- Consider how you would make $\mathcal{S}_1$ given $\mathcal{S}_0$:
  1. For each state x_0 in $\mathcal{S}_0$
  2. For each available action u_0 in state x_0
  3. Compute the next state x_1 using `p_next(x_0, u_0)` and add it to $\mathcal{S}_1$.
  4. Repeat this again to get $\mathcal{S}_2$ from $\mathcal{S}_1$ and so on
- You don't have to account for ghost-droids yet: Everything is deterministic
- The function should return a list of lists of `GameState` objects
- Avoid duplicated states in the same state space. I added a test for this situation.
- Check your solution using UNITGRADE

[3] `[ [a,b,c,..], [x,y,z,...], ...]`

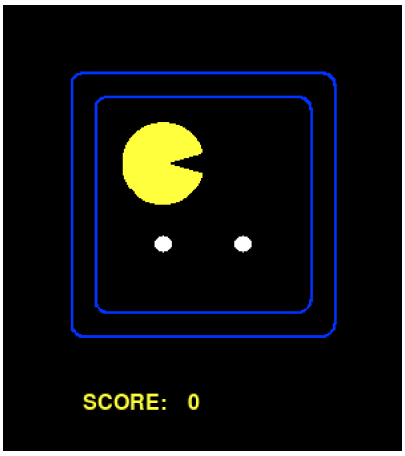

Figure 3: The state-counting map

Problem 10 *Shortest path*

You now have all the ingredients to find the datadiscs in the least amount of time. Do this by completing the function `shortest_path(map_layout, N)`. It should return two values (i) a list of optimal actions (ii) the list of states R2D2 will thereby traverse. Doing this, R2D2 should be able to find the optimal path in the no-ghost map shown in fig. 4.

Info:

- Solve the problem with dynamical programming DP
- To do so, build a DP-problem class corresponding to the problem. It is convenient to let the next state, x_{k+1}, be equivalent to the random noise disturbance as we did in the first problem of Exercise 2. See also [Her25, section 6.2.4].
- The functions in the DP problem class should be very short: The previous problem allowed you to define the state spaces, and the first problem allows you to define

$$f_k(x_k, u_k, w_k) = w_k, \text{ and } p_W(w_k|x_k, u_k) \text{ is equivalent to } \texttt{p_next(x_k, u_k)}$$

- Determine an appropriate cost-function, i.e. one that assigns a **higher** cost the **longer** it takes to win.
- First check the optimal cost agrees with what it should be, then focus on the optimal action/state sequences
- There is a problem from the first week which is relevant for showing how to simulate a policy and obtain the states/actions from a starting state.
- Check your solution using UNITGRADE

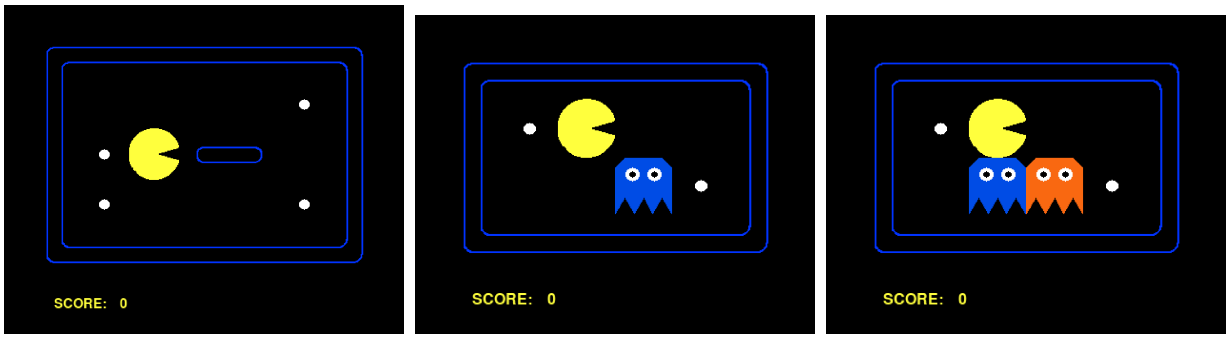

Figure 4: Example map with no ghosts (`datadiscs`), one ghost (`SS1tiny`) and two ghosts (`SS2tiny`)

2.3 One ghost-droid problems

In this problem R2D2 has to account for one ghost-droid (see map in fig. 4). We have to account for the randomness in the problem using the w_k-disturbance terms in the DP problem. To this end, remember that a ghost-droid selects between the available actions with uniform probability.

Problem 11 *Predict consequence of actions with one ghost*

Update the `p_next(x,u)` function to account for a single ghost-droid. The input it still an action, and the output is a dictionary where the keys are the states obtained after: (i) one deterministic move by Pacman and (ii) one random move of the ghost, and the values are their corresponding probability.

Info:

- **Remember probabilities should sum to 1**. Check this is the case as a sanity-check
- Since a ghost (usually) have three available actions, the dictionary will (usually) have three elements. The calculation is discussed in [Her25, section 6.2.4].
  1. Taking action u in state x brings the game to state x'.
  2. It is now the ghosts turn. For each action the ghost can take in state x', u', compute the next (final!) state x_{k+1} and the probability $p = \frac{1}{3}$.
  3. If this is the first time we see state x_{k+1}, let $p(x_k) \leftarrow 0$
  4. Update the probability as $p(x_{k+1}) \leftarrow p(x_{k+1}) + \frac{1}{3}$
  5. When done, return $p(x_{k+1})$ in the dictionary-representation, i.e. x_{k+1} are the keys.
- Check the output of the function manually and make sure it looks okay
- The code should be a natural extension of the code you have already written; i.e. the previous exercise should keep working, and the changes will properly be fairly minimal
- Remember the move of the ghost is made in the game state that arises after pacman has made his move. Use the functions mentioned in the `pacman_demo.py` file.
- Check your solution using Unitgrade

Next, we will compute the state-spaces $\mathcal{S}_0, \ldots, \mathcal{S}_N$ when there is one ghost-droid

Problem 12 *Possible future states with one ghost*

Update the function `get_future_states(x, N)`, which return a list of length $N + 1$ such that the k'th element correspond to $\mathcal{S}_k$ (the states R2D2 can reach in k steps starting from `x`, i.e. when it is R2D2's turn to move).

Info:

- The code should be a small update to your existing function assuming you used `p_next`. In fact, the code you have already written may in fact work
- If not, consider how you can use `p_next` to solve this problem
- Check your solution using UNITGRADE

Algorithm 1 Sketch of how `p_next` could be implemented

$p(f(x,u)) \leftarrow 1$ ▷ Probability of outcome after pacmans turn. $f(x,u)$ is the outcome of action u in game state x, `x.f(u)`.
for all ghosts **do**
  p' is a new, empty dictionary of probabilities.
  for all $x' \in p$ **do**
    $X \leftarrow$ By trying every action a' the ghost can take in x', compute all states that can occur after the ghosts move, i.e. $\{f(x', u') | u \in A(x')\}$
    ▷ The ghost is in state x', and the chance of that is $p(x')$. The chance it will take an action that brings it to x'' is therefore $\frac{1}{|X|}p(x')$. Therefore:
    for all $x'' \in X$ **do**
      $p'(x'') \leftarrow p'(x'') + \frac{1}{|X|}p(x')$ ▷ The chance is updated by the chance of getting to x', and then taking the action that brings us to x''
    end for
  end for
  $p \leftarrow p'$ ▷ After this ghost has moved, it is the next ghosts turn.
end for
p contains all possible states we can arrive at after the ghosts turn, plus their probability.

Problem 13 *Optimal one-ghost planning*

You now have all the ingredients to find the datadisc, while avoiding being caught by the ghost. To implement this, we assume that the reward is $g_N(x_N) = -1$ if the terminal state x_N is a won configuration and $g_N(x_N) = 0$ if it is lost, and that at other time steps $g_k(x_k, u_k, w_k) = 0$. In other words, the function you implement should just return the probability of winning within N steps; how soon it happens within the N steps is irrelevant.

Info:

- Solve this using a new DP model, nearly identical to the old one, and using backward-DP
- We are not adding a living-cost since we are only interested in the chance of success, not how long time it takes.
- I recommend you only compute each state-space once and store the result. They do not change, and your implementation can become very slow if you re-compute them many times.
- Check your solution using Unitgrade

2.4 Any-ghost planning

We are finally ready to tackle an arbitrary number of ghosts and still plan completely optimally. The most difficult part of this task is the following:

Problem 14 *Predict consequence of actions with several ghosts*

Update the `p_next(x,u)` function to account for any number of ghost-droids. I.e, in the case of two ghost-droids, the function is given a state and action, and returns a dictionary where the keys are the states obtained after (i) one deterministic move by pacman (ii) one (random) move by the first ghost and (iii) another random move by the second ghost, and the values are the probability of the resulting state.

Info:

- Focus on understanding what happens when e.g. `x` is the starting configuration and `u` is a specific action (such as going east). What can the ghosts do?
- Remember probabilities should sum to 1. Although not required, I recommend checking if this is the case automatically as a sanity-check to see if you are on the right track. I suspect most bugs will be found in this way.
- The ghosts take turns to move: First account for what the first ghost does (and which states that can lead to), then what the second ghost does (and what states that lead to)
- More details can be found in [Her25, section 6.2.4]. The best way to solve the problem is to think through how you would include the actions of a second ghost in the previous example, however, I have included a code listing of a possible solution in algorithm 1. Note that a recursive solution would be more elegant!
- Although the difference between 2 and 5 ghost implementation is not actually that big, your code will only be tested with 0, 1 or 2 ghosts.
- Check your solution using Unitgrade.

Next, verify you compute your state spaces correctly:

Problem 15 *Future states*

Update `get_future_states(x, N)` to work for any number of ghosts.

Info:

- If you used `p_next` in your implementation, it is very likely your implementation just works
- Check your solution using Unitgrade

If you pass all tests in unitgrade, it is time to move on to the final test: R2D2 should now be able to steal the plans for the death-star and evade the ghost-droids using optimally planning over N steps.

Problem 16 *Optimal planning*

Solve the missing optimal-planning problems for two ghost-droids to compute the chance R2D2s mission will succeed.

Info:

- Use same cost function as before
- Very likely, no additional code is necessary. If you find yourself writing a lot of code, you may be on the wrong track

References

[Her25] Tue Herlau. Sequential decision making. (Freely available online), 2025.